

NumPy Array Dimensions

1-D, 2-D, 3-D, and N-D Arrays | shape & size

A NumPy array can hold data in different numbers of **dimensions**. Two attributes help describe an array's structure:

- **shape** — the number of elements along each dimension, given as a tuple.
- **size** — the total count of all individual (primitive) elements in the array.

1. 1-D (One-Dimensional) Array

A 1-D NumPy array contains direct **primitive elements** (numbers, strings, etc.) with no nested lists inside it.

script.py

```
import numpy as np

arr1 = np.array([])
print(f"arr1:{arr1}, size:{arr1.size}, shape:{arr1.shape}")

arr2 = np.array([1])
print(f"arr2:{arr2}, size:{arr2.size}, shape:{arr2.shape}")

arr3 = np.array([1, 2])
print(f"arr3:{arr3}, size:{arr3.size}, shape:{arr3.shape}")

arr4 = np.array([10, 30, 30])
print(f"arr4:{arr4}, size:{arr4.size}, shape:{arr4.shape}")
```

Note: In the original code, every print statement referenced arr1 instead of arr2/arr3/arr4 — a copy-paste slip. The corrected version above prints each array's own size and shape.

Output

Array	Values	size	shape
arr1	[]	0	(0,)
arr2	[1]	1	(1,)
arr3	[1 2]	2	(2,)
arr4	[10 30 30]	3	(3,)

Note: For a 1-D array, shape is a tuple with one number — that number equals the size, since there is only one dimension.

2. 2-D (Two-Dimensional) Array

A 2-D NumPy array contains **1-D elements** — each row is itself a 1-D array, so the overall array is a list of lists.

```
script.py

import numpy as np

arr1 = np.array([
[]
])
print(f"arr1:{arr1}, size:{arr1.size}, shape:{arr1.shape}")

arr2 = np.array([
[10, 20, 30, 40],
[20, 30, 40, 50],
])
print(f"arr2:{arr2}, size:{arr2.size}, shape:{arr2.shape}")

arr3 = np.array([
[10, 20, 30, 40],
[20, 30, 40, 50],
[40, 50, 40, 50],
])
print(f"arr3:{arr3}, size:{arr3.size}, shape:{arr3.shape}")
```

Output

Array	Rows	size	shape
arr1	1 empty row	0	(1, 0)
arr2	2 rows of 4	8	(2, 4)
arr3	3 rows of 4	12	(3, 4)

Note: shape is now (rows, columns). arr2 has 2 rows and 4 columns, so shape = (2, 4) and size = $2 \times 4 = 8$.

3. 3-D (Three-Dimensional) Array

A 3-D NumPy array contains **2-D elements** — each "block" is itself a 2-D array (a list of rows), and the outer array is a list of those blocks.

script.py

```
import numpy as np

arr1 = np.array([
    [[1, 234, 80]],
])
print(f"arr1:{arr1}, size:{arr1.size}, shape:{arr1.shape}")

arr2 = np.array([
    [[1, 234, 80]],
    [[1, 234, 80]],
])
print(f"arr2:{arr2}, size:{arr2.size}, shape:{arr2.shape}")
```

script.py

```
arr3 = np.array([
    [[1, 234, 80]],
    [[1, 234, 80]],
    [[1, 234, 80]],
    [[1, 234, 80]],
])
print(f"arr3:{arr3}, size:{arr3.size}, shape:{arr3.shape}")

arr4 = np.array([
    [[1, 234, 80], [10, 20, 40]],
    [[1, 234, 80], [50, 60, 5]],
    [[1, 234, 80], [1, 3, 2]],
    [[1, 234, 80], [2, 5, 7]],
])
print(f"arr4:{arr4}, size:{arr4.size}, shape:{arr4.shape}")
```

Output

Array	Structure	size	shape
arr1	1 block, 1 row, 3 cols	3	(1, 1, 3)
arr2	2 blocks, 1 row, 3 cols	6	(2, 1, 3)
arr3	4 blocks, 1 row, 3 cols	12	(4, 1, 3)
arr4	4 blocks, 2 rows, 3 cols	24	(4, 2, 3)

Note: shape is now (blocks, rows, columns). For arr4: 4 blocks × 2 rows × 3 columns = 24, matching size = 24. Note the original code renamed the last array to arr3, overwriting the earlier arr3 — each new array here has been given its own name (arr1–arr4) to keep things clear.

4. N-D (N-Dimensional) Array

If a NumPy array contains elements nested deeper than 3 dimensions, we call it an **N-dimensional array** (n-D array). NumPy doesn't stop at 3-D — arrays can have as many dimensions as needed, limited only by memory.

```
script.py

# if a numpy array contains a numpy array having dimension
# greater than 3, we call it an N-dimensional numpy array
```

Note: The pattern is consistent at every level: a 1-D array holds numbers, a 2-D array holds 1-D arrays, a 3-D array holds 2-D arrays, and in general an N-D array holds (N-1)-D arrays. shape simply grows one more entry for each extra dimension, and size is always the product of every number in shape.

Quick Summary

Dimension	Contains	shape format	Example shape
1-D	Primitive elements (numbers, strings)	(n,)	(3,)
2-D	1-D arrays (rows)	(rows, cols)	(2, 4)
3-D	2-D arrays (blocks)	(blocks, rows, cols)	(4, 2, 3)
N-D	(N-1)-D arrays	(d1, d2, ..., dN)	(varies)

Note: Rule of thumb: size = product of all the numbers in shape. For example, shape (4, 2, 3) gives size = $4 \times 2 \times 3 = 24$.